Geometric algorithms

This methodology can be adopted to solve complex problems that are inherently geometric. It can be applied to solve problems concerning physical objects ranging from large buildings (design) and automobiles, to very large-scale integrated circuits (ICs).

But, you will soon see that even the most elementary operations (even on points) are computationally challenging. The interesting aspect is that some of these problems can readily be solved just by looking at them (and some others by applying the concepts in graph theory). If we resort to computational methods, we may have to go in for non-trivial methodologies.

This branch is relatively new and many fundamental algorithms are still being developed. Hence you can consider this as a potentially challenging and promising realm.

In this introductory piece, we'll restrict ourselves to the two-dimensional space. If you are able to properly define any point, then we can easily manage to include complex geometrical objects, say a line (as it is a pair of points connected by a straight line segment) or a polygon (defined by a set of points-array).

We can represent them by:

```
type point = record x,y: integer end;
     line = record pl, p2: point end;
```

It is quite easy to work with pictures compared to numbers, especially when it comes to developing a new design (algorithm) pattern. It is also very helpful while debugging the code.

Let's see a recursive program that will enable us to 'draw' a line by drawing the endpoints.

```
procedure draw(l: line) ;
 variable Δx, Δy: integer;
    p: point; 10,11: line;
 begin
 dot(l.pl.x,l.pl.y); dot(l.p2.x,l.p2.y);
 Δx:=l.p2.x-1.pl.x; Δy:=l.p2.y-1.pl.y;
 if (abs(Δx)>1) or (abs(Δy)>1) then
   begin
   p.x:=l.pl .x+Δx div 2; p.y:=l.pl .y+Δy div 2;
   l1.pl:=l.pl;   l.p2:=p; draw(l0);
   l2.pl:=p; l2.p2:=l.p2; draw(11);
   end ;
 end
```

You can see that there is a division of the space into two parts, joined by using line segments. You may stumble upon many algorithms where we will be converting geometric objects to points in a specific way. We can group them under the term 'scan-conversion algorithms'. To get a clear picture, you may write the pseudo code to check whether two lines are intersecting. (Hint: check for a common point.)

If you can't straight away do it, try this function to compute these lines and check whether they meet our condition:

```
function same_point(l: line; pl,p2: point): integer;
 variable Δx, Δy, Δxl, Δx2, Δyl, Δy2: integer;
 begin
 Δx:=l.p2.x-1.pl.x;    Δy:=l.p2.y-1.pl.y;
  Δxl :=pl .x-l.p1.x; Δyl :=pl.y-l.p1 .y;
  Δx2:=p2.x-1.p2.x; Δy2:=p2.y-1.p2.y;
 same_point:=(Δx*Δyl-Δy*Δxl)*(Δx*Δy2-Δy*Δx2)
 end;
```

If the quantity (Δx. Δyl – Δy. Δxl) is non-zero, we can say that pl is not on the line.

A problem for beginners

Here we are not trying to address a real problem! We will look at how to produce graphical output with the help of libraries. You might have drawn 'pictures' in BASIC while at school, but this is not that method. In fact, our intentions are different.

Let's define our problem: We need to draw a sphere with the help of a few straight lines.

We can use HoloDraw (see the resource links for more information) as the library for drawing the sphere and we will do the codes in Shell. We start by 'flattening' the sphere to a flat rectangular map.

As it is a sphere, we will meddle with the changes in terms of 'degrees'. We also need an input file for processing by the HoloDraw. (Before you proceed, download a copy of HoloDraw and untar it into a local directory. Also make sure that you have Perl installed.)

The input file, *sphere.draw*, will be quite akin to the following:

```
color=0 1 0
# draw a line around the sphere's equator
line: 0 0 1000, 360 0 1000
line: 0 45 1000, 360 45 1000
line: 0 -45 1000, 360 -45 1000

color=0 0 1

line: 0 90 1000, 0 -90 1000
line: 180 90 1000, 180 -90 1000

line: 30 90 1000, 30 -90 1000
line: 60 90 1000, 60 -90 1000
line: 90 90 1000, 90 -90 1000
line: 120 90 1000, 120 -90 1000
line: 150 90 1000, 150 -90 1000
line: 210 90 1000, 210 -90 1000
line: 240 90 1000, 240 -90 1000
line: 270 90 1000, 270 -90 1000
line: 300 90 1000, 300 -90 1000
```